Министерство образования и науки России
Санкт-Петербургский Государственный Технический Университет

УДК N .....
Инв. N ...

Рукопись одобрена кафедрой .....

Зав. кафедрой Черноруцкий И.Г.

РУКОПИСНЫЙ ФОНД
кафедры информационных и управляющих систем

Учебное пособие по курсу: "Системы программирования"

РАЗРАБОТКА ЭЛЕМЕНТОВ УЧЕБНОЙ СИСТЕМЫ ПРОГРАММИРОВАНИЯ. КОМПИЛЯТОР С АССЕМБЛЕРА

Для студентов кафедры инфомационных и управляющих систем
Семестр 10

Автор ..... Расторгуев В.Я.

С.-Петербург

2011 г.

Министерство образования и науки России
Санкт-Петербургский Государственный Технический Университет
Кафедра информационных и управляющих систем
Расторгуев В.Я.

РАЗРАБОТКА ЭЛЕМЕНТОВ УЧЕБНОЙ СИСТЕМЫ ПРОГРАММИРОВАНИЯ. КОМПИЛЯТОР С АССЕМБЛЕРА

Учебное пособие

С.-Петербург

2011 г.

АННОТАЦИЯ

Данное учебное пособие является методическим руководством при подготовке и проведению второй из трех лабораторных работ, направленных на разработку учебной системы программирования в составе:

- компилятора с языка высокого уровня,
- компилятора с АССЕМБЛЕРА,
- абсолютного или непосредственно-связывающего загрузчика, конструктивно объединенного с эмулятором и пошаговым отладчиком объектного (target) микропроцессора, работающего на платформе, состоящей из технологического (host) микропроцессора IBM PC и одной из операционных сред MS Windows или Linux.

Целью проведения этого цикла работ является более глубокое изучение лекционного материала курса и приобретение практических навыков в процессе разработки учебной системы программирования путем адаптации программ-макетов ее элементов. Каждая из работ цикла рассчитана при соответствующей домашней подготовке в среднем на восемь часов работы в лаборатории кафедры.

СОДЕРЖАНИЕ

1. Введение 5

2. Компилятор с АССЕМБЛЕРА. Тезисы лекций по курсу. . 6

2.1. Постановка задачи 6

2.2. Состав базы данных 12

2.3. Форматы элементов базы данных 13

2.3.1. Исходный текст 13

2.3.2. Счетчик адреса 14

2.3.3. Таблица машинных операций 14

2.3.4. Таблица псевдоопераций 15

2.3.5. Таблица символов 16

2.3.6. Таблица базовых регистров и адресов . . . 17

2.3.7. Объектный текст 18

2.4. Алгоритм компилятора 20

2.4.1. Алгоритм первого просмотра 20

2.4.2. Алгоритм второго просмотра 21

3. Рекомендации по порядку выполнения работы 22

4. Требования к содержанию отчета 23

5. Требования к комплектации, оформлению и предъявлению результатов курсовой работы 24

1. ВВЕДЕНИЕ

Данная лабораторная работа имеет своей целью получение практических навыков построения компилятора с языка АССЕМБЛЕР, являющегося одним из элементов системы программирования, образующих в совокупности следующий технологический конвейер:

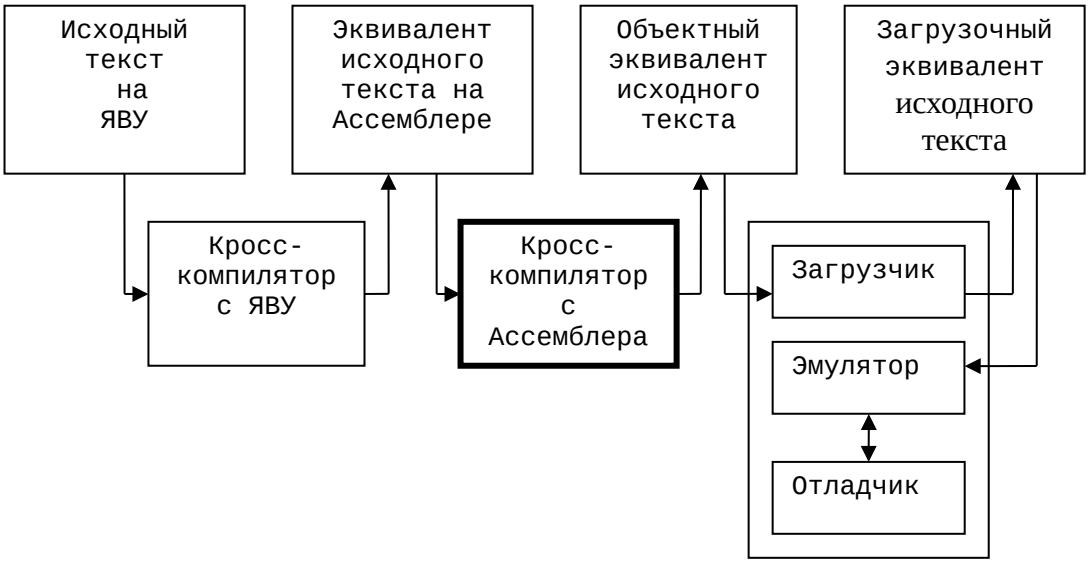


Рис.1. Состав учебной системы программирования

При этом предполагается то, что данная система программирования работает на технологической ЭВМ (IBM PC) и является по существу кросс-системой для объектной ЭВМ (IBM 370). В этой системе:

- в качестве языка высокого уровня (ЯВУ) выбран язык, образованный из подмножества языковых конструкций ПЛ1, а исходная программа готовится в виде текстового файла технологической ЭВМ с расширением *.pli,
- язык АССЕМБЛЕРА сформирован из языковых конструкций АССЕМБЛЕРА IBM 370, а ассемблеровский эквивалент исходной программы формируется в виде текстового файла технологической ЭВМ с расширением *.ass,
- объектный эквивалент исходной программы готовится в формате объектных файлов операционной системы ОС IBM 370 ЭВМ и хранится в виде двоичного файла технологической ЭВМ с расширением *.tex;
- загрузочный эквивалент исходной программы представляет собой машинный код IBM 370, запоминаемый в области ОЗУ технологической ЭВМ, являющейся зоной загрузки для эмулятора объектной ЭВМ.

2. КОМПИЛЯТОР С АССЕМБЛЕРА. ТЕЗИСЫ ЛЕКЦИЙ ПО КУРСУ

Проектирование компилятора с АССЕМБЛЕРА следует выполнять по типовому плану, состоящему из следующих этапов:

№ этапа	Содержание этапа
1	Постановка задачи.
2	Определение состава БД.
3	Определение форматов элементов БД.
4	Синтез алгоритма.
5	При необходимости выделение из схемы алгоритма фрагментов (модулей), нуждающихся в дальнейшей детализации.
6	Рекурсивное повторение этапов 1 – 5 для выделенных модулей.

2.1. Постановка задачи

Для построения эскизного варианта реализации компилятора с АССЕМБЛЕРА воспользуемся методом "сделай сам, зафиксируй способ решения и научи автомат".

Для ручного ассемблирования на язык машинных команд возьмем в качестве входных данных программу на АССЕМБЛЕРЕ, приведенную ниже в графе таблицы "исходный текст". Алгоритм этой программы написан на языке АССЕМБЛЕРА для решения следующей простейшей арифметической задачи:

Дано: A=3; B=4; C=5.
Найти: D=A+B-C.

Договоримся обрабатывать исходный текст построчно, записывая результат в графу "результат 1-го просмотра". При этом для каждой обрабатываемой строки исходного текста нам придется строить соответствующий машинный эквивалент, просматривая исходную строку слева направо, начиная с графы метка и заканчивая графой "операнд", заменяя при этом метку на относительный адрес, код операции (КОП) и операнды на их цифровые значения.

№	Исходный текст	Результат первого	Результат второго

строки				просмотра			просмотра		
	метка	КОП	операнды	Отн. адр.	КОП	операнды	Отн. адр.	КОП	операнды
1	EXAMP1	START	0	0			0		
2		BALR	RBASE,0	0	BALR’	?,0	0	BALR’	5,0
3		USING	*,RBASE	2			2		
4	BEGCALC	L	RRAB,A	2	L’	?,?(0,?)	2	L’	3,18(0,5)
5		A	RRAB,B	6	A’	?,?(0,?)	6	A’	3,22(0,5)
6		S	RRAB,C	10	S’	?,?(0,?)	10	S’	3,26(0,5)
7		ST	RRAB,D	14	ST’	?,?(0,?)	14	ST’	3,30(0,5)
8		BCR	15,RVIX	18	BCR’	15,?	18	BCR’	15,14
9	A	DC	F’3’	20	Память для A=3		20	Память для A=3	
10	B	DC	F’4’	24	Память для B=4		24	Память для B=4	
11	C	DC	F’5’	28	Память для C=5		28	Память для C=5	
12	D	EQU	*	32			32		
13		DS	F	32	Память для D		32	Память для D	
14	RBASE	EQU	5						
15	RRAB	EQU	3						
16	RVIX	EQU	14						
17		END							

Так мы и сделаем, но со следующими малыми отступлениями:

- во-первых, чтобы не усложнить прочтение преобразованного текста, договоримся заменять код операции на цифровое значение условно, т.е., оставляя в полученном эквиваленте исходной строки прежнее символическое значение кода операции, дополненное апострофом справа, и будем, тем не менее, считать, что замена на цифровое значение проведена;
- во-вторых, для сохранения читабельности мы будем проводить условную замену и для операндов исходной команды, применяя вместо машинного (цифрового) формата - мнемонический машинно-ориентированный формат.

Дальнейшее рассмотрение покажет, что полную обработку исходного текста за один единственный его просмотр выполнить в общем случае нельзя по причинам, указанным ниже. Исходный текст необходимо будет просмотреть два раза. Поэтому далее обсудим фазы обработки исходного текста (просмотры) с привязкой к номеру просмотра исходного текста, на котором она производится.

П е р в ы й п р о с м о т р

Первая строка исходного текста никакого исполняемого кода не порождает, т.к. является директивой START компилятора с АССЕМБЛЕРА, которая своим операндом устанавливает счетчик относительного адреса равным нулю.

Вторая строка порождает код команды BALR RR-формата, длиной в два байта, в которой КОП имеет цифровое шестнадцатеричное значение

0x05 и которое после условной замены оставляется нами в прежнем символическом виде с апострофом справа - "BALR’". Из двух же операндов этой команды заменить на цифровое значение мы сможем только второй, который является самоопределенным и равным 0. Для замены первого операнда нужно знать какое цифровое значение связано с меткой RBASE. Эта метка определяется в строке 14 исходного текста, которую мы в момент обработки строки 2, еще не исследовали. Поэтому максимум, что мы можем сделать, это отложить определение первого операнда на следующий (второй) просмотр исходного текста, а в строке 2 эквивалента на месте первого операнда поставим знак вопроса – “?”. Последним действием скорректируем счетчик относительного адреса новым значением 2, равным длине команды BALR.

Третья строка содержит директиву компилятора с АССЕМБЛЕРА USING, операнды которой сообщают компилятору, что, начиная с этой строки, ему в своей работе следует считать, что в Регистре Общего Назначения (РОН), номер которого определен через значение метки RBASE, содержится абсолютный адрес, соответствующий текущему значению относительного адреса, равному 2. Счетчик относительного адреса данная директива не меняет.

При обработке четвертой строки пояснение требуется только в части построения эквивалента второго операнда. Здесь задача сводится к представлению абсолютного адреса, соответствующего метке A, триадой вида:

D,(X,B), где:

- D - смещение относительно базового адреса, содержащегося в РОН с номером B;
- X - номер РОН-а, содержащего значение индекса;
- B - номер РОН-а, содержащего значение базового абсолютного адреса.

В нашем случае полное построение такой триады невозможно, т.к. мы в данный момент еще не исследовали строку 9, содержащую определение адреса и значения поля, помеченного именем A, и строку 15 с определением метки RRAB. Точно мы можем высказаться только об индексном регистре, который, судя по формату второго операнда, просто не используется (иначе второй операнд был бы в исходной строке записан в формате A(X)). Неиспользование индексного регистра кодируется в эквиваленте четвертой строки значением X=0. Вместо остальных элементов триады ставим знак вопроса, оставляя определение их значений до следующего (второго) просмотра исходного текста. Последним действием при обработке четвертой строки будет коррекция счетчика относительного адреса на длину команды в четвертой строке, равной четырем байтам, что даст в результате значение 6.

Пятая, шестая и седьмая строки обрабатываются подобно четвертой строке, а восьмая - подобно второй.

Обработка девятой строки сводится к:

- резервированию четырех байтов для хранения переменной, помеченной именем A, с занесением в них значения 3,
- запоминанию того факта, что с именем A связан относительный адрес 20,
- коррекция счетчика относительного адреса на длину переменной A, равной четырем байтам.

Десятая и одиннадцатая строки обрабатываются подобно девятой строке.

Двенадцатая строка содержит директиву АССЕМБЛЕРА EQU, операнд которой заставляет компилятор связать с именем D текущее значение счетчика относительного адреса, равное 32.

Тринадцатая строка является псевдокомандой АССЕМБЛЕРА, при обработке которой следует зарезервировать четыре байта для размещения переменной с именем D без ее начальной инициализации каким-либо значением и увеличить текущее значение счетчика относительного адреса на длину этой переменной, т.е. на четыре байта.

Директива АССЕМБЛЕРА EQU в строке 14 заставляет компилятор связать с именем RBASE абсолютное значение 5.

Пятнадцатая и шестнадцатая строки обрабатываются подобно строке 14.

Семнадцатая строка, являясь директивой АССЕМБЛЕРА END, завершает данный просмотр.

Перед тем, как прокомментировать действия по обработке исходного текста на втором просмотре, подведем итоги первого просмотра, во время которого мы собрали следующие данные об исходном тексте, полезные для второго просмотра, а именно:

- нами отобрано множество всех меток и имен полей, определенных в исходном тексте программы и предназначенных для пометки отдельных команд и полей, отведенных для хранения переменных,
- для каждой метки или имени мы теперь установили, связанное с ними абсолютное (число) или переместимое (относительный адрес) выражение,
- мы установили длину программы в байтах, которая равна текущему значению счетчика относительного адреса в момент завершения первого просмотра.

Приступим теперь ко второму просмотру, записывая получаемые результаты в графу "результат 2-го просмотра".

В т о р о й п р о с м о т р

Теперь алгоритм обработки исходного текста на втором просмотре

очевиден. Если говорить коротко, то он сводится к последовательному просмотру строк исходного текста и замене знаков вопроса в операндах на рассчитываемые во время второго просмотра цифровые значения. Детализируем сказанное построчно.

Во второй строке доопределим первый операнд RBASE, заменив его на теперь уже известное нам после первого просмотра значение, равное 5.

В четвертой строке в коррекции нуждается второй операнд, причем сразу в двух элементах триады:

- смещении,
- номере базового регистра.

Теперь после первого просмотра мы располагаем всеми необходимыми данными для вычисления числовых значений этих элементов. Перед тем, как сделать это, вспомним, что во время первого просмотра нам стало известно то, что:

- в строке 14 было сделано указание считать имя RBASE связанным с числом 5,
- в строке 2 было сделано указание считать РОН с номером RBASE (т.е. с номером 5), связанным с относительным адресом 2, и объявить этот РОН в качестве базового регистра,
- в строке 9 было сделано объявление переменной A, размещенной по относительному адресу 20 со значением 5.

Теперь очевидно, что для того, чтобы во втором операнде четвертой строки сформировать ссылку на метку A, нам достаточно выбрать базовый регистр номер 5 и смещение, равное 18.

В стоках с пятой по седьмую расчет элементов триад для формирования второго операнда делается так же, как и для строки 4.

Окончательное формирование второго операнда в строке 8 делается подобно тому, как это было выполнено для строки 2.

Подведем итоги по результатам нашей попытки "ручного" ассемблирования и попытаемся на их основе сформулировать эскизные контуры алгоритма проектируемого компилятора с АССЕМБЛЕРА.

Анализ наших действий во время "ручного" ассемблирования позволяет сделать вывод о том, что всю задачу ассемблирования удобно решать в два этапа, на каждом из которых следует производить просмотр исходного текста.

При этом на **первом** этапе просмотра следует вычислить значения для всех меток и запомнить их в специальной таблице символов (TSYM). Для этого потребуется:

- Построчно сканируя исходный текст (ISXTXT), корректировать счетчик адреса (CHADR) (на длину кода машинного эквивалента),

запоминая его значение в таблице TSYM вместе с идентификатором метки в момент ее определения.

- В случае обнаружения в процессе сканирования исходного текста псевдокоманд: START, DC, DS, EQU, и END обрабатывать их только в части, касающейся коррекции CHADR и определения значений меток, с записью последних в TSYM.

Для программной реализации этого потребуются в общем случае следующие агрегатированные данные, являющиеся элементами базы данных (БД) компилятора:

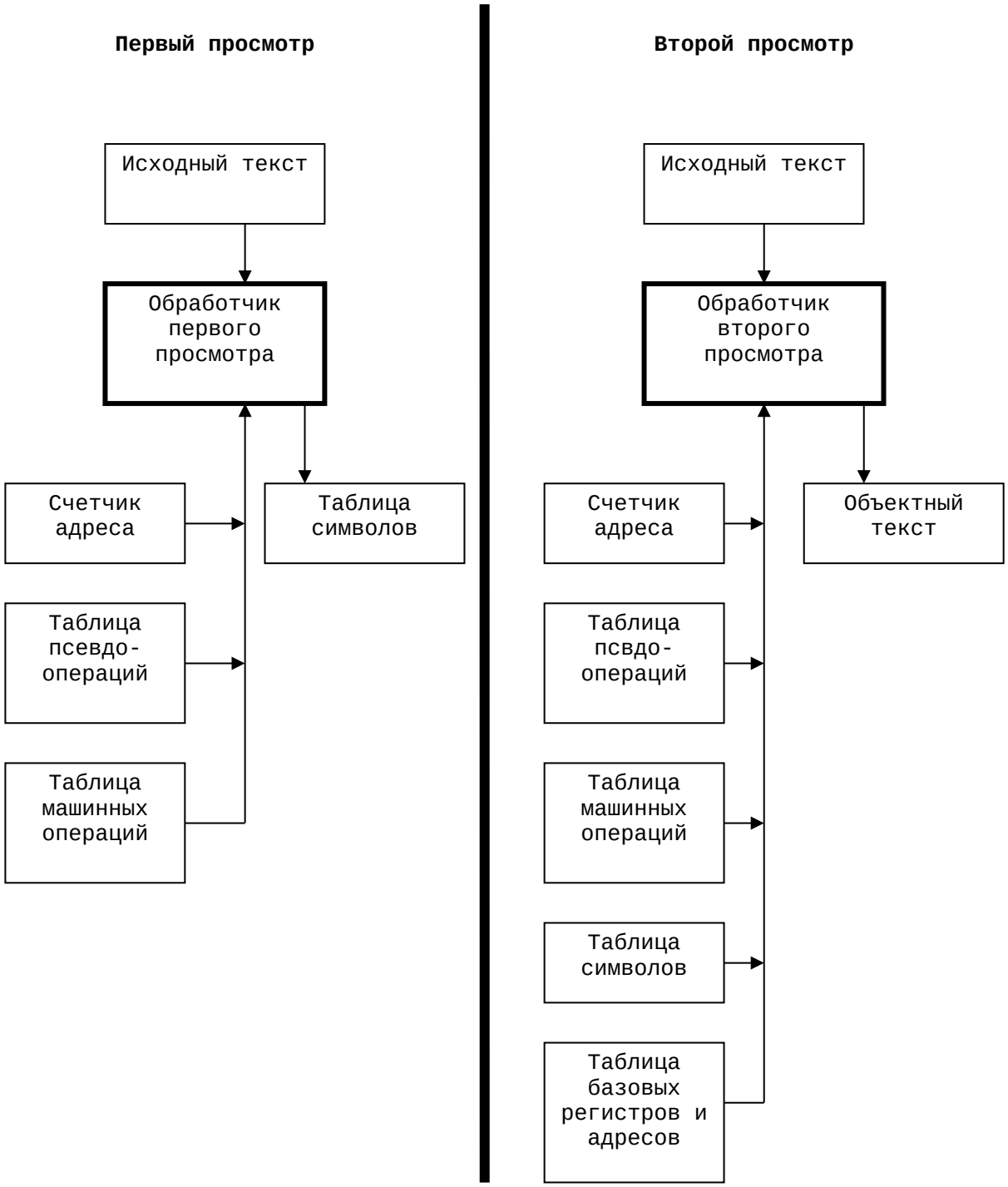
- исходный текст,
- счетчик адреса,
- таблица символов,
- таблица характеристик псевдоопераций,
- таблица характеристик машинных операций.

На **втором** просмотре следует, используя исходный текст и результаты первого просмотра создать объектный эквивалент исходной программы. Для этого потребуются следующие элементы БД:

- исходный текст,
- счетчик адреса,
- таблица символов,
- таблица характеристик псевдоопераций,
- таблица характеристик машинных операций,
- таблица базовых адресов (хранящая содержимое РОН-ов, используемых в качестве регистров базы),
- объектный текст (объектный эквивалент исходного текста).

2.2. Состав БД

В соответствии с результатами эскизной проработки алгоритма компилятора, сделанной нами, БД компилятора и схема ее включения в информационные потоки может быть представлена ниже следующим образом:



2.3. Форматы элементов БД

Для каждого из элементов БД перед программной реализацией следует выбрать:

- имя элемента, которое м.б. использовано как идентификатор соответствующей переменной (скалярной, индексированной или агрегатированной),
- конкретный вид скалярного (табличного) представления.

Описывая далее результаты нашего выбора табличного представления того или иного элемента БД, мы будем в заголовке каждой графы таблицы сверху вниз последовательно указывать:

- содержательное описание назначения поля элемента БД,
- идентификатор поля элемента БД,
- длину поля элемента,
- тип поля элемента в соглашениях языка программирования Си.

Описание будет сопровождаться примерами заполнения строк обсуждаемой таблицы.

2.3.1. Исходный текст

И д е н т и ф и к а т о р

ISXTXT

В и д п р е д с т а в л е н и я

Исходный текст после его загрузки из файла в ОЗУ м.б. представлен в виде следующей таблицы, которая имеет своим программным аналогом в языке Си массив структур:

Метка	Мнемокод операции	Операнды	Комментарии
METKA	OPERAC	OPERAND	COMM
8 байтов	5 байтов	12 байтов	52 байта
char[8]	char[5]	char[12]	char[52]
EXAMP	START BALR	0 5,0	Установить счетчик адреса CHADR=0 Занести в РОН 5 значение CHADR + длина команды BALR, равное 2

П р и м е ч а н и я:

- длина каждой строки исходного текста равна 80 символов,
- каждое поле строки исходного текста имеет свою заданную фиксированную длину, указанную в заголовке таблицы,
- различные поля одной и той же строки разграничены разделительным пробелом.

2.3.2. Счетчик адреса

И д е н т и ф и к а т о р

CHADR

В и д п р е д с т а в л е н и я

Счетчик адреса должен быть представлен как скалярная переменная с разрядностью не менее 24 битов (согласно предельному значению адресного пространства объектной IBM 370, которое равно 16 мб.). Учитывая то, что целые переменные в предполагаемой технологической ЭВМ IBM PC имеют разрядность либо 8, либо 16, либо 32 битов, выбираем 32-х разрядный вариант.

2.3.3. Таблица машинных операций

И д е н т и ф и к а т о р

TMOP

В и д п р е д с т а в л е н и я

Таблица машинных операций м.б. представлена в следующем виде, который имеет своим программным аналогом в языке Си массив структур:

Мнемокод машинной операции	Код машинной операции	Длина машинной операции	Указатель на программу обработки данной машинной операции с учетом ее формата на 1-м просмотре	Указатель на программу обработки данной машинной операции с учетом ее формата на 2-м просмотре
MNCOD	CODOP	DLOP	BXPROG[0]	BXPROG[1]
5 байтов	8 битов	1 байт	4 байта	4 байта
char[5]	unsigned char	char	int (*)()[]	int (*)()[]
“BALR” “BCR”	0x05 0x07	2 2	FRR FRR	SRR SRR

П р и м е ч а н и я:

- максимально-возможная длина мнемокода машинной операции в АССЕМБЛЕРЕ ЕС ЭВМ равна 5 символам,
- длина команды ЕС ЭВМ м.б. равна либо 2, либо 4, либо 6 байтам и, следовательно, для ее хранения в таблице символов достаточно отвести целую переменную с минимальной разрядностью в 1 байт,
- все форматы машинных операций в ЕС ЭВМ кодируются двумя символами, например: RR, RX, SS, и т.д. При компиляции каждая команда, в зависимости от ее формата, будет обрабатываться на первом проходе одной из подпрограмм: FRR, FRX, FSS и т.д., а на втором - соответственно: SRR, SRX, SSS и т.д.

2.3.4. Таблица псевдоопераций

И д е н т и ф и к а т о р

ТРОР

В и д п р е д с т а в л е н и я

Таблица псевдоопераций м.б. представлена в следующем виде, который имеет своим программным аналогом в языке Си массив структур:

Мнемокод псевдооперации	Указатель на программу обработки данной псевдооперации с учетом ее формата на 1-м просмотре	Указатель на программу обработки данной псевдооперации с учетом ее формата на 2-м просмотре
MNCOD	BXPROG[0]	BXPROG[1]
5 байтов	4 байта	4 байта
char[5]	int (*)()[]	int (*)()[]
“START” “USING”	FSTART FUSING	SSTART SUSING

П р и м е ч а н и я:

- максимально-возможная длина мнемокода псевдооперации в АССЕМБЛЕРЕ ЕС ЭВМ равна 5 символам,
- имя подпрограммы обработки каждой псевдооперации X на первом проходе начинается с символа F (First), а на втором – с S (Second). Последующие символы совпадают с наименованием псевдокоманды.

2.3.5. Таблица символов

И д е н т и ф и к а т о р

TSYM

В и д п р е д с т а в л е н и я

Таблица символов м.б. представлена в следующем виде, который имеет своим программным аналогом в языке Си массив структур:

Символ метки из помеченной строки исходной программы	Значение, сопоставляемое символу	Длина, сопоставляемая символу	Признак переместимости символа
IMSYM	ZNSYM	DLSYM	PRPER
8 байтов	4 байта	1 байт	1 байт
char[8]	unsigned int	unsigned char	char
EXAMP1	0	36	‘R’
BEGCALC	2	4	‘R’
A	20	4	‘R’
B	24	4	‘R’
C	28	4	‘R’
D	32	4	‘R’
RBASE	5	1	‘A’
RRAB	3	1	‘A’
RVIX	14	1	‘A’

П р и м е ч а н и я:

- для идентификации имен (меток) будем применять идентификаторы, состоящие из не более чем из 8-ми символов,
- имена (метки) должны быть отнесены либо к абсолютным (имеющим признак переместимости, равный 'A', и значение, независящее от адреса загрузки), либо к переместимым (имеющим признак переместимости, равный 'R', и значение, зависящее от адреса загрузки),
- учитывая, что в качестве значения переместимого имени (метки) берется значение соответствующего ему адреса, выбираем для его хранения разрядность, совпадающую с разрядностью счетчика адреса, т.е. 32 бита,
- для всех имен (меток), соответствующих строкам исходного текста, порождающим машинный код, в качестве длины выбирается длина этого кода. Для всех же имен (меток), соответствующих строкам исходного текста, непорождающим машинный код, в качестве длины условимся выбирать значение 1.

2.3.6. Таблица базовых регистров и адресов

И д е н т и ф и к а т о р

TBASR

В и д п р е д с т а в л е н и я

Таблица базовых регистров и адресов м.б. представлена в следующем виде, который имеет своим программным аналогом в языке Си массив структур:

Значение базового адреса, записанного в РОН, выбранного базовым	Признак выбора РОН в качестве базового регистра
SMESH	PRDOST
4 байта	1 байт
unsigned int	char
-	‘N’
-	‘N’
-	‘N’
-	‘N’
2	**‘Y’**
-	‘N’
-	‘N’
-	‘N’
-	‘N’
-	‘N’
-	‘N’
-	‘N’
-	‘N’
-	‘N’
-	‘N’

П р и м е ч а н и я:

- в качестве базового регистра м.б. выбран любой из регистров за исключением РОН с номером 0,
- в строках таблицы, соответствующих РОН, выбранных в программе в качестве базовых, в графе PRDOST устанавливаются признаки доступности, равные 'Y'. А для всех остальных - 'N',
- одновременно с установкой признака доступности в графу SMESH заносится значение счетчика адреса, сопоставленное данному базовому регистру.

2.3.7. Объектный текст

И д е н т и ф и к а т о р

OBJTEXT

В и д п р е д с т а в л е н и я

Таблица объектных текстов м.б. представлена в следующем виде, который имеет своим программным аналогом в языке Си массив 80-ти байтовых строк, каждая из которых м.б.отнесена к одному из следующих трех типов (ESD-строка, TXT-строка, END-строка).

ESD-строка

//// ////	Тип строки	//// ////	Имя программы	//// ////	Начальный адрес	//// ////	Длина программы	//////////// ////////////
	POLE2		IMPR		ADPRG		DLPRG	
1 байт	3 байта	12 байтов	8 байтов	1 байт	3 байта	1 байт	3 байта	48 байтов
	char[3]		char[8]		char[3]		char[3]	
	“ESD”		“EXAMP1”		0		36	

П р и м е ч а н и я:

- для ESD-строки в поле "тип строки" устанавливается значение "ESD",
- в поле "имя программы" переписывается значение метки псевдооперации START, которое отождествляется с именем всей программы,
- в поле "начальный адрес" переписывается значение операнда псевдооперации START,
- в поле "длина программы" заносится длина программы в байтах,
- здесь и далее поля строки, в заголовках которых проставлено "/.../", заполняются постоянными значениями, неизменяемыми программой-макетом и, поэтому, исключаются из пояснения.

TXT-строка

////	Тип	////	Относительный	////	Длина	////	Операнд	////////////

////	строки	////	адрес операции	////	операнда	////		//////////
	POLE2		ADOP		DLNOP		OPER	
1 байт	3 байта	1 байт	3 байта	2 байт	2 байта	4 байт	56 байта	8 байтов
	char[3]		char[3]		char[2]		char[56]	
	“TXT”		0		2		“\x05\x50”	

П р и м е ч а н и я :

- для TXT-строки в поле "тип строки" устанавливается значение "TXT",
- в поле "адрес операции" устанавливается значение CHADR, соответствующее той операции из программы, которая указана далее в графе "операнд" TXT-строки и длина которой указана в графе "длина операнда" TXT-строки.

END-строка

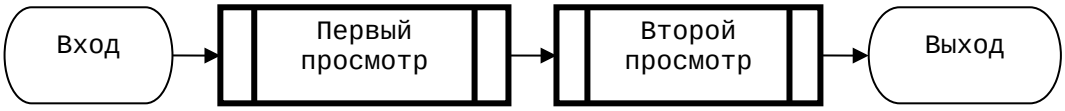
//// ////	Тип строки	// //
	POLE2	
1 байт	3 байта	76 байтов
	char[3]	
	“END”	

П р и м е ч а н и я :

- для END-строки в поле "тип строки" устанавливается значение "END".

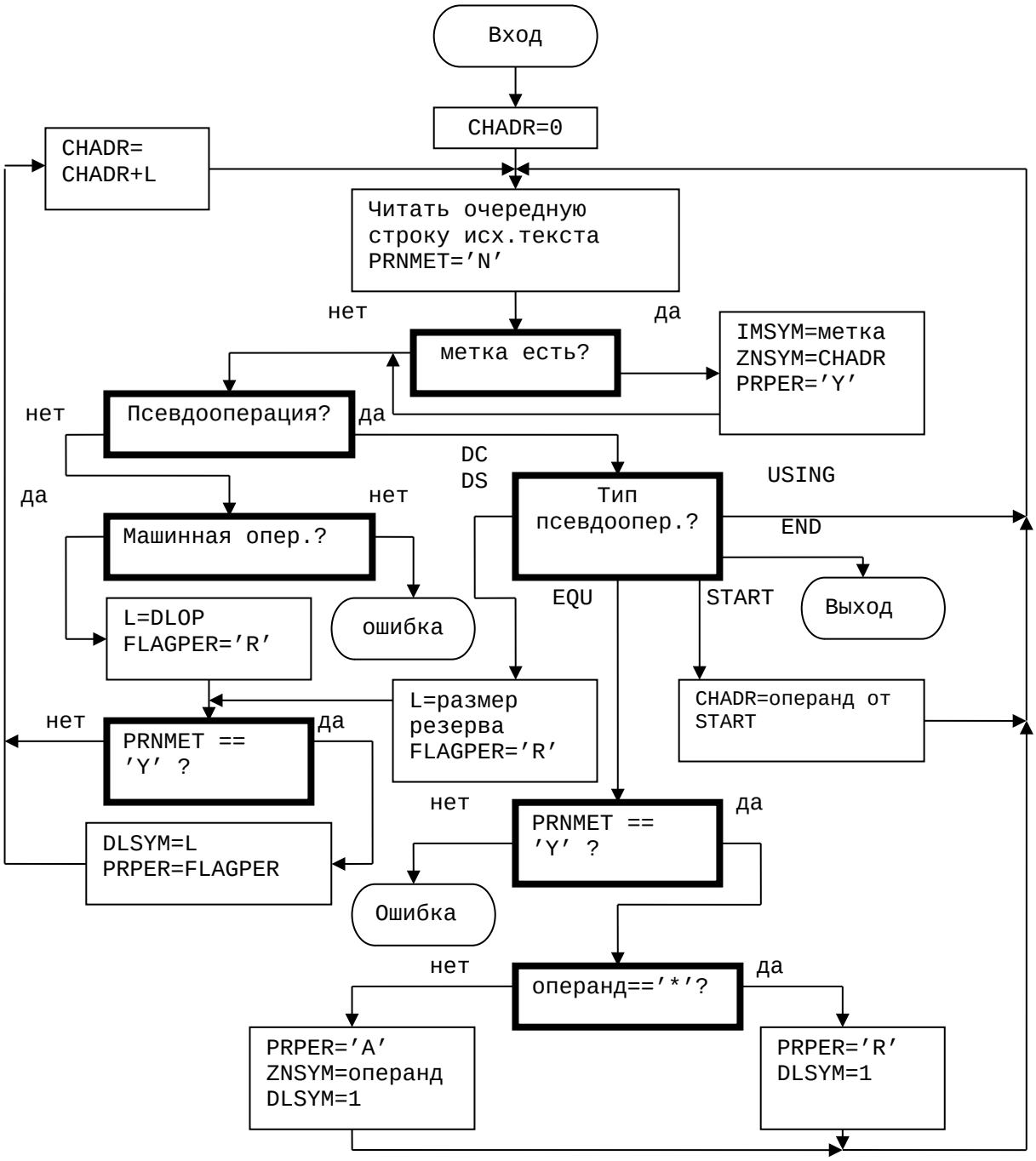
2.4. Алгоритм компилятора

Согласно выводам по результатам постановки задачи (пп. 2.1.) алгоритм компилятора м.б. представлен следующим образом:

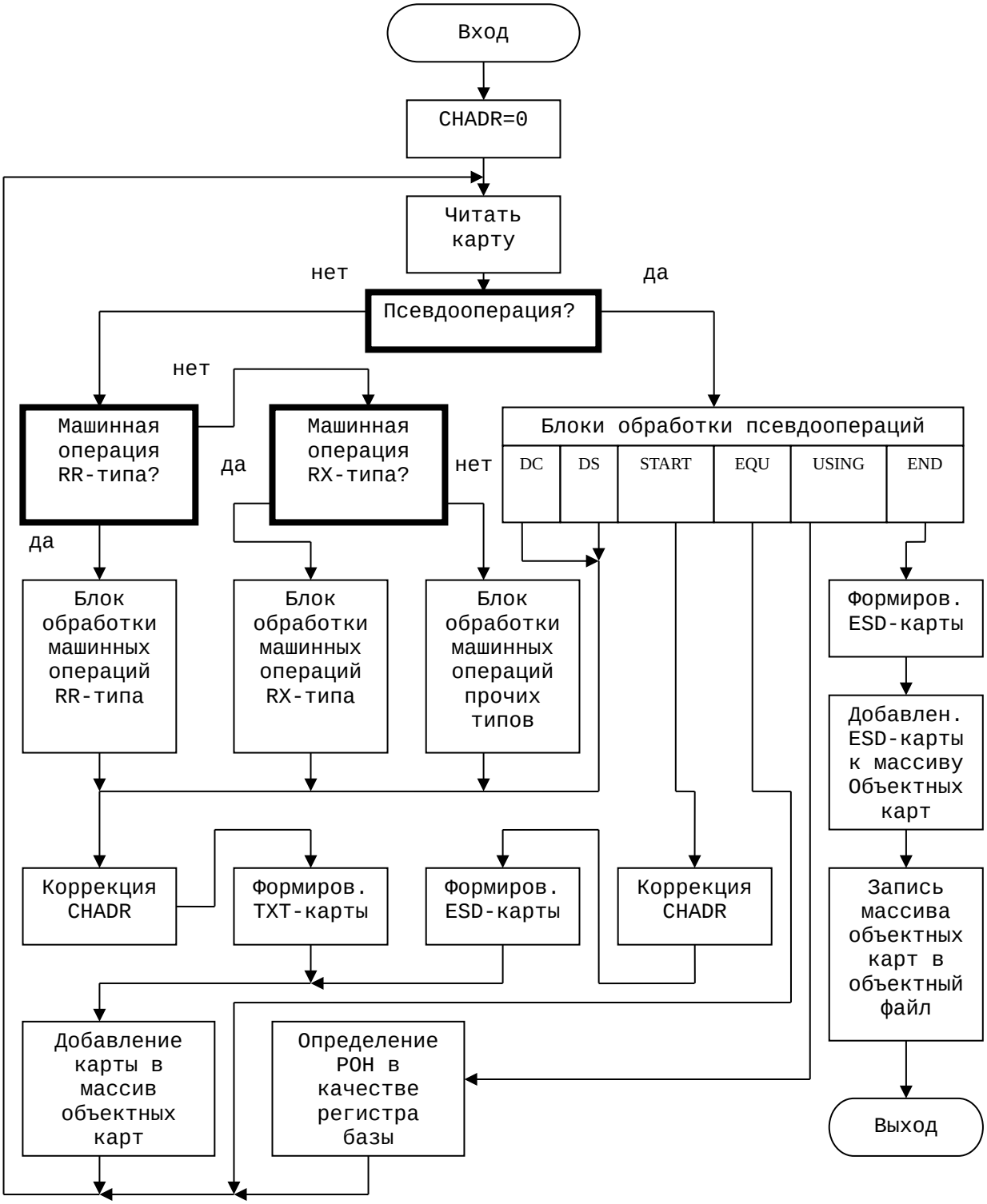


Детализируем эту макро-блок-схему так, как это показано ниже.

2.4.1. Алгоритм первого просмотра



2.4.2. Алгоритм второго просмотра



3. РЕКОМЕНДАЦИИ ПО ПОРЯДКУ ВЫПОЛНЕНИЯ РАБОТЫ

Данная лабораторная работа предполагает использование в качестве начальной основы отлаженной и работающей программы-макета компилятора с АССЕМБЛЕРА IBM 370, с ограниченным набором языковых конструкций, достаточным для написания законченной правильной программы контрольного примера. Программа-макет компилятора с АССЕМБЛЕРА написана на алгоритмическом языке Си.

Студенту предлагается расширить первичный набор языковых конструкций контрольного примера до уровня, позволяющего оттранслировать без ошибок пример, содержащийся в его личном задании. При этом необходимо:

- ознакомиться с исходными текстами программы-макета и контрольного примера, на который настроена программа-макет компилятора с АССЕМБЛЕРА,
- проверить работоспособность программы-макета компилятора с АССЕМБЛЕРА путем проведения компиляции контрольного примера,
- вручную построить ассемблеровский эквивалент исходного текста примера, содержащегося в личном задании, и выделить из него множество языковых конструкций, отсутствующих в ассемблеровском эквиваленте исходного текста контрольного примера,
- спроектировать перечень модификаций элементов БД и подпрограмм программы-макета компилятора с АССЕМБЛЕРА с целью обеспечения компиляции ассемблеровского эквивалента примера из личного задания,
- ввести модификации в исходный текст программы-макета компилятора с АССЕМБЛЕРА, снабдив измененные места исходного текста построчным комментарием, и записать модифицированный вариант в свой личный директорий на сервере,
- выполнить отладку модифицированной программы-макета компилятора с АССЕМБЛЕРА,
- выполнить отработку своего ассемблеровского эквивалента личного задания с помощью модифицированной программы-макета компилятора с АССЕМБЛЕРА,
- составить отчет по результатам работы.

4. ТРЕБОВАНИЯ К СОДЕРЖАНИЮ ОТЧЕТА

Отчет должен соответствовать рекомендациям по оформлению студенческих курсовых работ, размещенных на сайте университета, и в общем случае должен содержать следующие разделы:

- описание примера из личного задания, включая его ассемблеровский эквивалент (указать формулировку личного задания),
- постановка задачи модификации программы-макета компилятора с АССЕМБЛЕРА (описание выбранного способа модификации программы-макета),
- перечень и пояснения модификаций состава и (или) структуры БД программы-макета компилятора с АССЕМБЛЕРА,
- перечень и пояснения модификаций алгоритма программы-макета компилятора с АССЕМБЛЕРА (допустимыми являются описания в виде: блок-схемы, текста на Си, а в очевидных случаях обычным текстом),
- выводы по результатам работы (указать позитивные и негативные моменты, проявившиеся в процессе работы, с общей личной оценкой проделанной работы).

5.ТРЕБОВАНИЯ К КОМПЛЕКТАЦИИ, ОФОРМЛЕНИЮ И ПРЕДЪЯВЛЕНИЮ РЕЗУЛЬТАТОВ КУРСОВОЙ РАБОТЫ

К результатам курсовой работы относятся:

- Отчеты в бумажном исполнении,
- Отчеты в электронном исполнении,
- Исходные тексты компилятора с ЯВУ,
- Исходные тексты компилятора с АССЕМБЛЕРА,
- Исходные тексты загрузчика.

Сводный отчет по курсовой работе “Системы программирования”, выполненной студентом, должен:

- быть укомплектован из отчетов по всем элементам разработанной учебной системы программирования,
- по своей структуре соответствовать рис.2,
- быть подготовлен как в бумажном, так и в электронном вариантах исполнения,
- быть передан преподавателю после завершения курсовой работы в момент защиты результатов курсовой работы.

Работы по отдельным элементам разрабатываемой учебной системы программирования проводятся в течение семестра. Отчеты по каждому элементу:

- составляются по завершению работы над его реализацией,
- предъявляются преподавателю в течении семестра,
- включаются в сводный отчет к концу семестра.

Исходные тексты элементов учебной системы программирования вместе с электронным вариантом сводного отчета оформляются и передаются преподавателю в виде архивного файла, рекомендуемый состав папок и файлов которого приведен на рис.3. В названии архива **Var_XX**, вместо **XX** следует использовать номер варианта своего задания на курсовую работу.

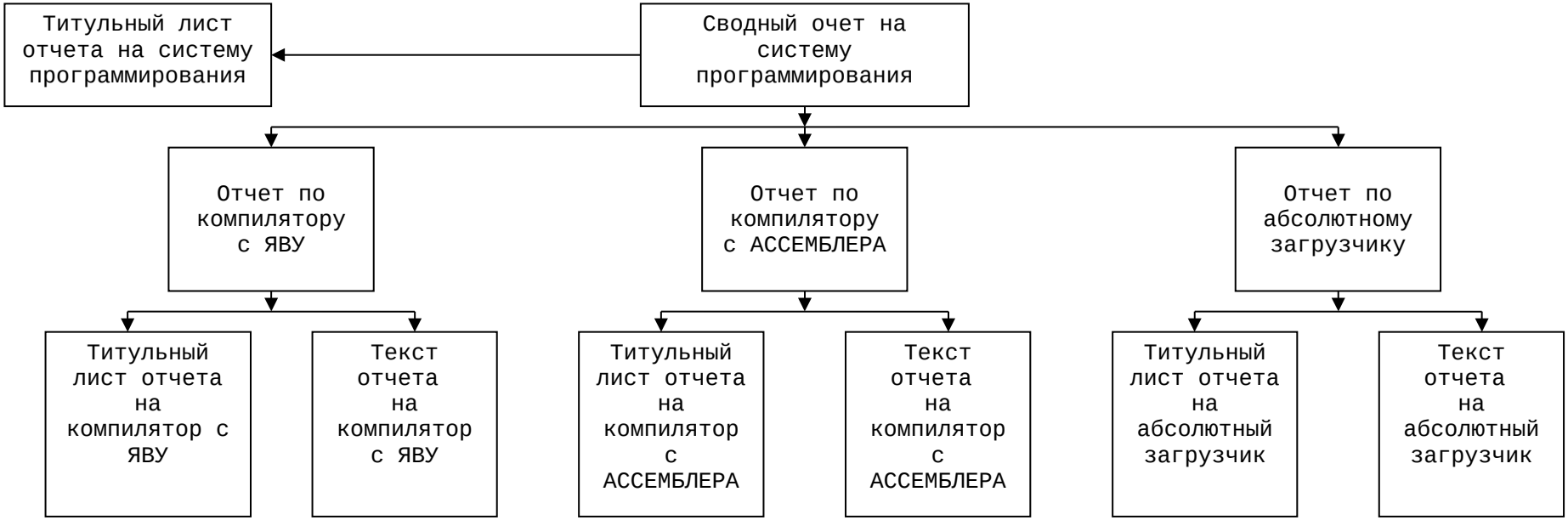


Рис.2. Структура сводного отчета по курсовой работе

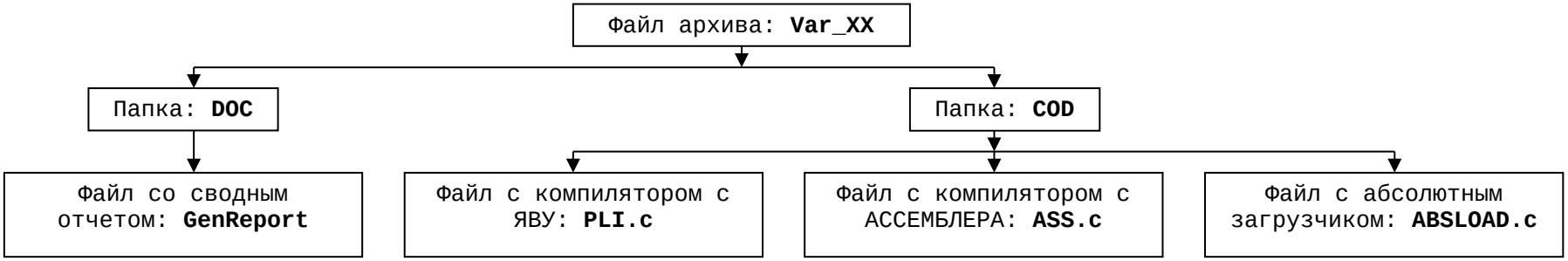


Рис.3. Рекомендуемый состав архива